



自动驾驶轨迹预测——VectorNet

模型实现讲解与实践（基于nuPlan）

主讲人 蒲宇



大家好，我叫蒲宇，今天由我来做一次关于VectorNet模型实现和在nuplan数据集上实践的分享

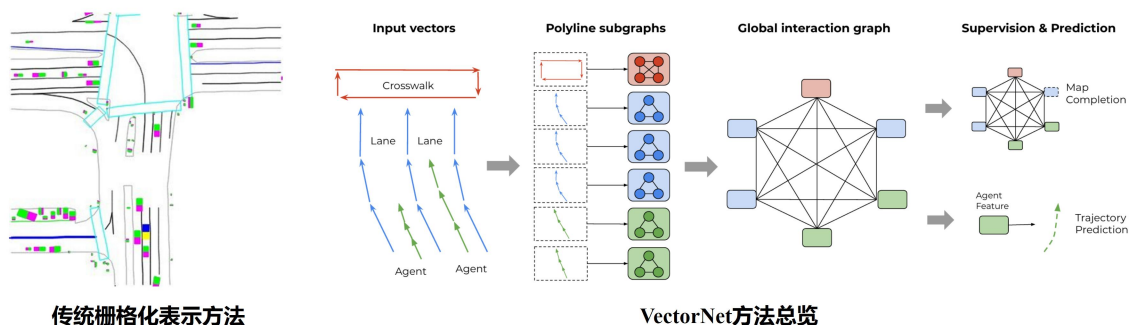
- **第一部分：背景与预备知识**
- **第二部分：VectorNet方法**
 - 2.1 历史轨迹与地图元素的特征表示
 - 2.2 基于GNN的局部特征提取
 - 2.3 基于self-attention的全局特征提取
- **第三部分：在nuPlan数据集上的实践**

<https://arxiv.org/pdf/2005.04259>



将从以下三个方面展开

背景



1. 首次展示如何使用矢量化场景元素和智能体动态信息来做轨迹预测；
2. 提出了分层图网络 VectorNet 及其节点补全辅助任务；
3. 在内部和公开数据集上评估所提出的方法，证明VectorNet能取得更好的性能指标，且具有更小模型尺寸和更低的运算量

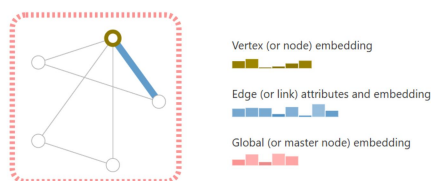
首先是背景部分，vectornet是2020年的工作，放在4年后的今天来看各方面性能指标其实没那么突出，那什么我们还是去看呢？

首先是因为vectornet确实是一项里程碑式的工作，也启发了后续很多工作，其次呢是相对一些比较新的工作上手门槛更低，可以帮助我们更顺滑地入门自动驾驶轨迹预测。

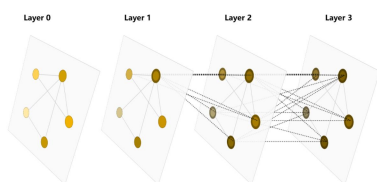
在vectornet之前做轨迹预测的主流方式是将信息栅格化地表示出来，根据预先定义好的规则将输入渲染成一张图片并通过CNN做特征提取，这样能够work但并不高效，很明显的一个原因就是像高精地图、agent轨迹这样的信息本身是高度结构化的，却又转换成了图像这样的高维信息再用CNN来提取特征似乎是有点多此一举，另外CNN的感受野有限难以获取更大范围内的交互。因此vectornet首次展示如何使用矢量化场景元素和智能体动态信息来做轨迹预测，并通过实验证明在更小模型尺寸

和更低运算量的前提下能够达到更好的性能指标，以上是背景

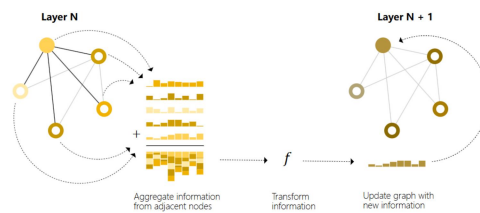
GNN图神经网络



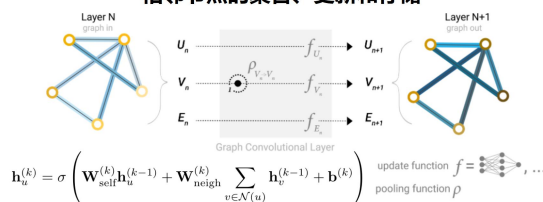
向量信息可以存储在每个图节点或边。



GNN通过网络各层从周围的节点聚合信息示意



相邻节点的聚合、更新和存储



GCN架构示意图

<https://distill.pub/2021/gnn-intro/>

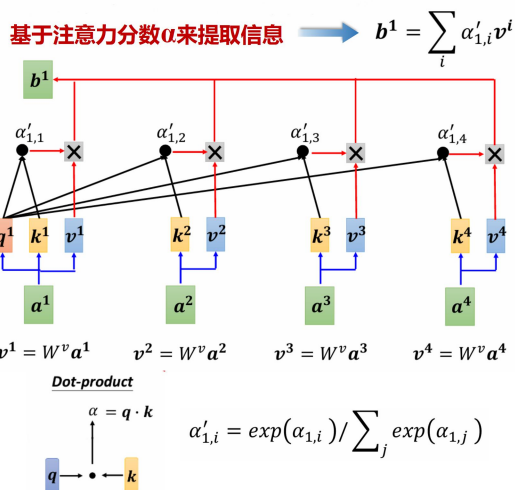
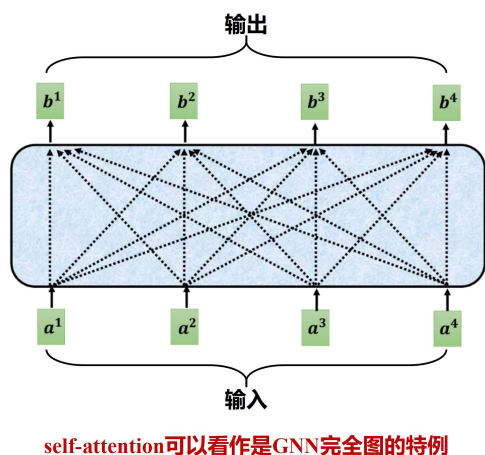
在正式开始讲vectornet模型实现之前，考虑到可能有部分同学对深度学习没那么熟悉，所以这里再啰嗦几句简要地介绍以下后续我们会用到的关键模块，首先是GNN图神经网络。

对于图这个数据结构计算机相关的同学应该都不陌生，由节点和边构成，每个节点和边都可以用一个向量来表示，在某些应用场景下也会有一个全局节点这样的一个虚拟节点存在。

图神经网络和卷积神经网络类似都是一种特征提取的途径，不一样的是CNN处理的是图像数据，而GNN处理的则是图这样的数据结构。具体来说输入一张图，GNN根据每个节点的邻居信息更新自身的信息，不改变图的结构输出新的图。

具体细节可以看左边这两张图，首先根据边的拓扑信息找到目标节点的邻居节点，然后聚合邻居节点的信息，最后通过神经网络更新得到最终结果，下面的这个公式描述了这一过程： $\mathbf{W}$ 是可学习的网络权重，这里的求和符号表示聚合不一定是求和还可以是平均或最大值等， $\mathbf{b}$ 是偏置项， ρ 是激活函数，以上是GNN最简单的模型实现，更多信息可以参考这篇distill博客，目前这些信息在vectornet中够用了

Self-Attention自注意力机制



<https://youtu.be/hYdO9CscNes?si=MsODE2eiOhqI5ORA>

然后是self-attention自注意力机制，self-attention的输入是一组向量，根据输入向量之间的关联性输出新的一组向量，从这张图很容易让人联想到GNN，这不就是每一对不同的顶点都有一条边相连的完全图的嘛，实际上在vectornet中也是这么做的。具体来看self-attention的计算过程，首先由一组权重矩阵将每个输入向量得到各自的QKV映射，其中Q表示query查询、K和V分别表示key和value就是我们常用的字典键值对，向量Q分别和所有向量K做内积得到对应的注意力分数 α ，求的所有 α 后执行

一次Softmax归一化，最后根据归一化后的注意力分数对所有的value做加权求和得到对应query的输出，可以发现self-attention中待训练的参数只有qkv各自对应的权重矩阵 w ，并且它是共享的。

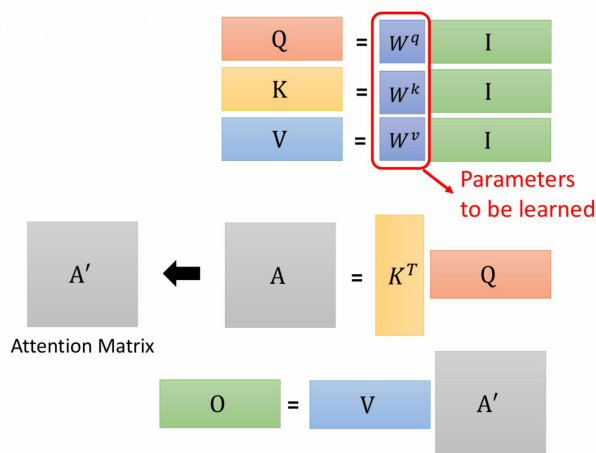
这是self-attention最简单的一种形式，不过对于学习vectornet已经足够了，更多像multi-head、mask等有关信息可以参考李宏毅老师的完整课程

Self-Attention自注意力机制

$$\begin{aligned}
 q^i &= W^q a^i & q^1 q^2 q^3 q^4 &= W^q \begin{matrix} a^1 & a^2 & a^3 & a^4 \end{matrix} \\
 & & Q & & I \\
 k^i &= W^k a^i & k^1 k^2 k^3 k^4 &= W^k \begin{matrix} a^1 & a^2 & a^3 & a^4 \end{matrix} \\
 & & K & & I \\
 v^i &= W^v a^i & v^1 v^2 v^3 v^4 &= W^v \begin{matrix} a^1 & a^2 & a^3 & a^4 \end{matrix} \\
 & & V & & I
 \end{aligned}$$

$$\begin{matrix}
 \alpha_{1,1} & \alpha_{2,1} & \alpha_{3,1} & \alpha_{4,1} \\
 \alpha_{1,2} & \alpha_{2,2} & \alpha_{3,2} & \alpha_{4,2} \\
 \alpha_{1,3} & \alpha_{2,3} & \alpha_{3,3} & \alpha_{4,3} \\
 \alpha_{1,4} & \alpha_{2,4} & \alpha_{3,4} & \alpha_{4,4}
 \end{matrix}
 =
 \begin{matrix}
 k^1 \\
 k^2 \\
 k^3 \\
 k^4
 \end{matrix}
 \begin{matrix}
 q^1 & q^2 & q^3 & q^4
 \end{matrix}$$

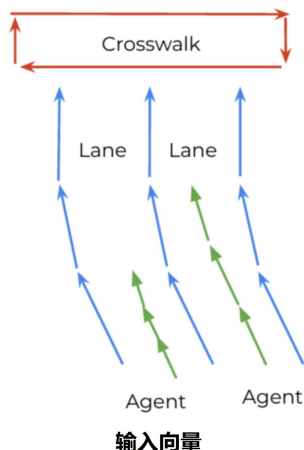
A
 K^T
 Q



并行处理大大提高了计算效率

<https://youtu.be/hYdO9CscNes?si=MsODE2eiOhq15ORA>

此外还有一点要说明的是，self-attention或者说transform之所以在目前大模型中广泛应用其中很重要的一个原因是它可以高效地并行计算，刚才我们所说的计算过程并不一定是串行的，而是可以像这样把多个向量写成矩阵形式然后交给底层经过硬件加速的矩阵计算库得到结果，从这里的示意图可以看到每个计算过程都是可以并行的



对于地图元素**等距离采样**，对于agent轨迹**等时间间隔**进行采样，每个向量具体表示如下：

$$\mathbf{v}_i = [\mathbf{d}_i^s, \mathbf{d}_i^e, \mathbf{a}_i, j]$$

其中 $\mathbf{d}_i^s$ 和 $\mathbf{d}_i^e$ 是向量的起始点和终点，可以用 (x, y) 或 (x, y, z) 表示； $\mathbf{a}_i$ 和向量的属性相关联，如agent/地图元素类型、轨迹的时间步或当前车道线向量的限速等； j 是折线 $\mathcal{P}_j$ 的整型id，表示 $\mathbf{v}_i \in \mathcal{P}_j$ ， $\mathbf{v}_i$ 作为子图 $\mathcal{P}_j$ 的节点特征。

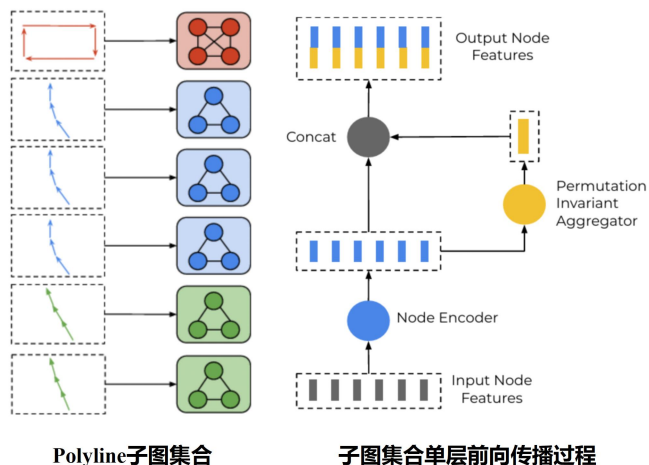
特别的，VectorNet是典型的agent-centric的工作，**所有向量表示需要变换到待预测的目标agent当前时刻的局部坐标系下！**

好，接下来就正式进入到vectornet模型实现的部分，本部分会按照原始论文的顺序和内容展开，首先环境信息和agent历史轨迹的向量化表示

前面我们提到过，本身地图元素和历史轨迹就是非常结构化且具体的信息，所以将其转换为向量表示是比较自然的。简单来说就是对于地图元素**等距离采样**，对于agent历史轨迹等实践间隔采样，每个向量具体表示形式如下， $\mathbf{d}_s$ 和 $\mathbf{d}_e$ 分别表示向量的

起点和终点，可以是 (x, y) 或 (x, y, z) 或者加入旋转信息，如 (x, y, θ) ； $\mathbf{a}$ 则与向量属性相关联，比如agent/地图元素类型、轨迹的时间步或当前车道线的限速等； j 是折线也是子图集合的id，相同id j 的向量属于同一子图，比如同一个agent的历史轨迹，或是同一条车道线。从上述的描述来看，其实大家会意识到这给出了一个整体的形式，但具体每个元素怎么表示并没有标准答案。比如很容易想到可能航向角朝向信息对预测未来轨迹很有帮助，但其实加不加上影响没想象中那么大，包括属性也行，这块

大家可以动手多去尝试实验看看。特别的，vectornet是典型的agent-centric的工作，上述所有向量化的表示都是放在待预测agent当前时刻的局部坐标系下的，否则训练收敛得很慢甚至模型直接不work了



1. 输入向量经过编码 g_{enc} 得到 $g_{enc}(v_i^l)$
2. $g_{enc}(v_i^l)$ 聚合得到 $\phi_{agg}\{g_{enc}(v_i^l)\}$
3. $g_{enc}(v_i^l)$ 和 $\phi_{agg}\{g_{enc}(v_i^l)\}$ 拼接得到输出

$$\mathbf{v}_i^{(l+1)} = \varphi_{rel}\left(g_{enc}(\mathbf{v}_i^{(l)}), \varphi_{agg}\left(\left\{g_{enc}(\mathbf{v}_j^{(l)})\right\}\right)\right)$$

DETAILS:

- Node Encoder使用共享权重的MLP
- MLP后接layer normalization和relu
- 聚合操作采用maxpooling

针对处理好且划分好的向量会首先通过GNN做一次特征提取，这步操作是针对每个子图也就是同一组折线分别进行的，所以只是在局部信息层面的特征提取。另外在上一页ppt中我们只提到了节点的表示，没有说边要怎么设置，在论文中提到每个子图都是完全图，也就是每个子图中的节点都和其他所有节点相连，但其实从我们的直觉来看这似乎不怎么合理，可能按顺序相连更符合逻辑一点，毕竟时间和空间都是连续的，这个也可以分别训练对比然后一下效果。

节点和边准备好，GNN的输入就有了，接下来看在vectornet中GNN模块具体的实现。对照着这张图来，首先所有输入节点向量经过一个MLP的编码器，并加layer normalization和relu激活函数，然后兵分两路，一路保持不动，一路经过maxpooling最大值的聚合操作，再将二者拼接起来，所以经过一次操作后节点的维度是会翻倍的，比如输入节点向量的维度是8输出节点的维度就变成16了。然后整个subgraph就是这样一个module的叠加

SubGraph代码实现

```
class SubGraph(nn.Module):
    def __init__(self, in_channels, num_subgraph_layers=3, hidden_unit=64):
        super(SubGraph, self).__init__()
        self.num_subgraph_layers = num_subgraph_layers
        self.layer_seq = nn.Sequential()

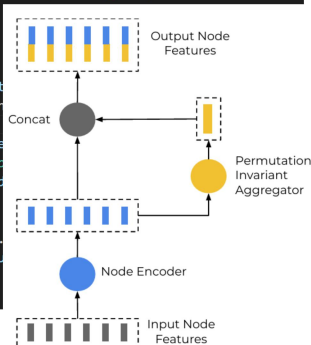
        for i in range(num_subgraph_layers):
            self.layer_seq.add_module(
                f'glnp_{i}',
                GraphLayerProp(in_channels, hidden_unit)
            )
            in_channels *= 2

    def forward(self, sub_data):
        x = sub_data.x.to(torch.float32)
        edge_index = sub_data.edge_index

        for _, layer in self.layer_seq._modules.items():
            if isinstance(layer, GraphLayerProp):
                x = layer(x, edge_index)

        sub_data.x = x
        out_data = max_pool(sub_data, edge_index)
        out_data.x = out_data.x / (out_data.x.size(0) ** 0.5)

        return out_data
```



```
from torch_geometric.nn import MessagePassing

class GraphLayerProp(MessagePassing):
    def __init__(self, in_channels, hidden_unit=64):
        super(GraphLayerProp, self).__init__(agg='max')

        self.mlp = nn.Sequential(
            nn.Linear(in_channels, hidden_unit),
            nn.LayerNorm(hidden_unit),
            nn.ReLU(),
            nn.Linear(hidden_unit, in_channels)
        )

        for layer in self.mlp:
            if isinstance(layer, nn.Linear):
                nn.init.kaiming_normal_(
                    layer.weight,
                    nonlinearity='relu'
                )

    def forward(self, x, edge_index):
        x = self.mlp(x.to(torch.float32))

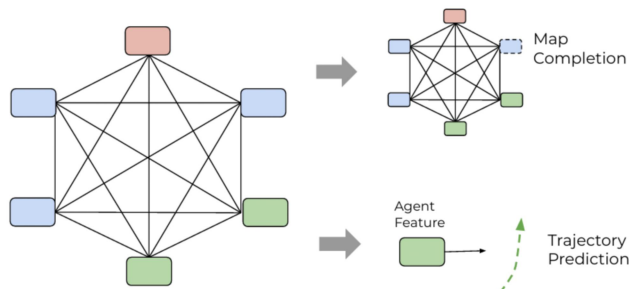
        return self.propagate(
            edge_index.to(torch.long),
            size=(x.size(0), x.size(0)),
            x=x
        )

    # 重载MessagePassing中的函数，在propagate中调用
    def update(self, aggr_out, x):
        return torch.cat([x, aggr_out], dim=1)
```

一起来看下代码实现，可以看到subgraph的主体就是多层GraphLayerProp的堆叠，最后会对每个子图执行max_pool的操作作为最终折线级别的特征输出

具体看下GraphLayerProp的实现，它继承自torch_geometric.nn模块的MessagePassing类，并且聚合方式设置为max最大值。首先进来先经过一个单隐藏层的MLP，然后就是propagate函数，在这个函数中做了通过最大值聚合和拼接两个操作，在这个函数中会用到在下面重载的update函数，原MessagePassing类的update不会做拼接这个操作。像这里的num_subgraph_layers有几层、hidden_unit mlp隐藏层单元数，包括这里mlp隐藏层层数这些都是

待正定的模型超参数，在论文的实验部分有消融实验对比不同参数设置对模型精度的印象，这里的默认值3和64就是实验中效果最好的一组超参数



全局交互图 & 预测

$$\{p_i^{(l+1)}\} = \text{GNN}(\{p_i^{(l)}\}, \mathcal{A})$$

论文 $\mathcal{A}$ 假设为**完全图**，使用self-attention实现GNN：

$$\text{GNN}(\mathbf{P}) = \text{softmax}(\mathbf{P}_Q \mathbf{P}_K^T) \mathbf{P}_V$$

$\mathbf{P}$ 是节点特征矩阵， $\mathbf{P}_Q, \mathbf{P}_K^T, \mathbf{P}_V$ 分别是其线性投影

$$\mathbf{v}_i^{\text{future}} = \varphi_{\text{traj}}(\mathbf{p}_i^{(L_t)})$$

Context	Node Compl.	In-house dataset				Argoverse dataset			
		DE@1s	DE@2s	DE@3s	ADE	DE@1s	DE@2s	DE@3s	ADE
none	-	0.77	0.99	1.29	0.92	1.29	2.98	5.24	2.36
map	no	0.57	0.81	1.11	0.72	0.95	2.18	3.94	1.75
map + agents	no	0.55	0.78	1.05	0.70	0.94	2.14	3.84	1.72
map	yes	0.55	0.78	1.07	0.70	0.94	2.11	3.77	1.70
map + agents	yes	0.53	0.74	1.00	0.66	0.92	2.06	3.67	1.66

消融实验

只针对局部信息提取特征肯定是不够，接下来我们把目光放远一点，看怎么处理全局信息。在全局信息处理时，我们把每组折线视作一个图的节点，从更宏观的角度看问题。

而至于边的设计，**vectornet**并没有人工设计一套规则去生成，而是简单地作为完全图来处理，这里可能会有疑问说自车未来几秒内的轨迹怎么会和全局可能几百米外的地图或agent有关系呢？有两点，一是在处理输入时我们就可以把全局信息限制在一个合理的范围，比如60m或者80m；另外关联性可以交给注意力分数来决定。就像前面说的，**self-attention**是完全图GNN的一种特例，最后**self-attention**的输出再通过MLP解码得到预测轨迹。这里需要说明的是，前面我们强调了

输入的向量必须转到目标agent当前时刻的局部坐标系下，所以我们只能针对目标Agent的节点做解码获取预测轨迹，不能在一次推理中同时获取到所有agent。如果所有agent解码获取预测轨迹，给对应真值强行训练，大概率loss是无法收敛的。

另外在论文中还提到了一个辅助补全任务，来估计节点间的交互，不过从后面的消融实验来看这个任务对精度的影响很小，而且也不是**vectornet**的核心，因此本次分享不会涉及这部分内容



GlobalGraph代码实现

```
class SelfAttentionLayer(nn.Module):
    def __init__(self, in_channels, global_graph_width=64):
        super().__init__()
        self.queryFC = nn.Linear(in_channels, global_graph_width)
        self.keyFC = nn.Linear(in_channels, global_graph_width)
        self.valueFC = nn.Linear(in_channels, global_graph_width)

        nn.init.kaiming_normal_(self.queryFC.weight)
        nn.init.kaiming_normal_(self.keyFC.weight)
        nn.init.kaiming_normal_(self.valueFC.weight)

    def forward(self, x):
        # Assuming x is of shape (B, N, D), where B is the batch size,
        # N is the number of items in the sequence, and D is the features dimension.
        p_query = F.relu(self.queryFC(x)) # (B, N, D_k)
        p_key = F.relu(self.keyFC(x)) # (B, N, D_k)
        p_value = F.relu(self.valueFC(x)) # (B, N, D_v)

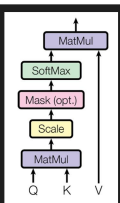
        # Perform batch matrix multiplication between p_query and the transpose of p_key
        # The shapes are (B, N, D_k) and (B, D_k, N) -> (B, N, N)
        query_result = torch.bmm(p_query, p_key.transpose(1, 2))

        # Scale query_result to avoid gradients vanishing with high dimensions
        query_result = query_result / (p_key.size(-1) ** 0.5)

        # Apply softmax to the last dimension to get the attention weights
        attention = F.softmax(query_result, dim=-1)

        # Now we perform batch matrix multiplication between attention weights and p_value
        # The shapes are (B, N, N) and (B, N, D_v) -> (B, N, D_v)
        output = torch.bmm(attention, p_value)

        # Add the residual connection
        return output + p_query
```



VectorNet backbone

```
class VectornetGNN(nn.Module):
    def __init__(
        self,
        in_channels,
        out_channels,
        num_subgraph_layers=3,
        subgraph_width=64,
        global_graph_width=64,
        traj_pred_mlp_width=64
    ):
        super(VectornetGNN, self).__init__()
        self.polyline_vec_shape = in_channels * (2 ** num_subgraph_layers)

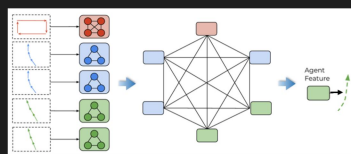
        self.subgraph = SubGraph(in_channels, num_subgraph_layers, subgraph_width)
        self.self_attn_layer = SelfAttentionLayer(
            self.polyline_vec_shape, global_graph_width)
        self.traj_pred_mlp = TrajPredMLP(
            global_graph_width, out_channels, traj_pred_mlp_width)

    def forward(self, data):
        subgraph_out = self.subgraph(data)

        self_attn_input = subgraph_out.x.view(
            -1, config.NUM_GRAPH, subgraph_out.num_features)

        self_attn_out = self.self_attn_layer(self_attn_input)
        agent_feature = self_attn_out[:, 0, :]
        mlp_input = agent_feature.contiguous().view(-1, agent_feature.shape[2])
        prediction = self.traj_pred_mlp(mlp_input)

        return prediction
```



具体看下代码实现，可以看到在GlobalGraph层待学习的权重只有计算QKV的全连接层，前向推理的过程就和我们前面背景知识里讲述的一致了，这放一张attention is all you need论文中self-attention机制计算过程的图对照看

首先是输入x分别经过三个全连接层得到QKV，Q和K两个矩阵相乘做一步缩放再经过softmax做归一化得到全量的注意力分数矩阵，这里的mask是针对时序信号的在globalgraph中不需要，注意力分数矩阵和矩阵V相乘就得到self-attention的输出，其实这里再堆叠几层也是可以的，不过论文中说明了是用的一层我们就先按论文里来吧。最后将attention的输出和V矩阵相加形成一个残差连接，这也是在深度学习中很经典的做法了。最后把所有的部分串起来就完成了vectornet，

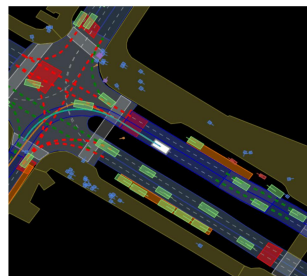
向量化的表示在模型输入前的预处理完成，输入首先经过subgraph做polyline级别的局部特征提取，接着是基于attention机制的全局交互特征提取，最后通过全连接层解码得到输出。

□ nuPlan简介

nuPlan是**世界第一个针对自动驾驶规划方法测试的开源数据集**（发布于2021年），收集了波士顿、匹兹堡、拉斯维加斯和新加坡这4个城市收集了大约1300小时的驾驶数据。

官网：<https://www.nuscenes.org/nuplan>

API仓库：<https://github.com/motional/nuplan-devkit>

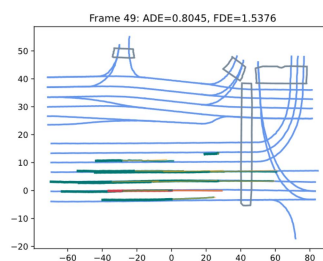


□ 注意事项

1. 数据集中存在部分异常数据需剔除
2. 标签应按**增量形式**给出



Our predicted trajectories are parameterized as **per-step coordinate offsets**, starting from the last observed location.



完成了vectornet的主要实现，具体效果如何，还是得在数据集跑了才知道，实践出真知嘛。之所以用nuplan是因为深蓝学院《[自动驾驶预测与决策规划](#)》这门课程除了有预测的内容还有决策规划这块的内容，nuplan是21年发布的第一个针对自动驾驶规划方法的开源数据集，而且同时提供开环和闭环仿真评测，场景数据丰富API也比较完善，不管是数据的交互还是像这样的一些可视化工具。我们需要通过API拿到需要的数据然后转换为模型要求的输入，这里有两点需要注意，一是和所有深度学习训练任务一样都需要做数据清洗，避免脏数据喂到模型里影响性能，nuplan也一样对于agent轨迹特别是追踪他车的轨迹少量存在异常的情况需要剔除出去；第二点是，真值标签要在局部坐标系下且以增量形式给出，这个在论文中有说明可能会有同学漏掉。这样做的目的是至少能让模型的输出像一条轨迹，而不是一条杂乱无章的折线，可以加速收敛。所以在拿到模型输出后我们也还需要做两次转换包括增量转为绝对坐标和坐标系的转换才拿到最终的预测结果


```
scenario_mapping = ScenarioMapping(get_scenario_map(), 0.5)
builder = NuPlanScenarioBuilder(config.DATA_PATH, config.MAP_PATH, sensor_root,
                                db_files, config.MAP_VERSION, scenario_mapping=scenario_mapping)
# filtering scenario
scenario_filter = ScenarioFilter(*get_filter_parameters(
    scenarios_per_type, total_scenarios, shuffle_scenarios))
worker = SingleMachineParallelExecutor(use_process_pool=True)
# get scenarios
scenarios = builder.get_scenarios(scenario_filter, worker)
```

使用nuplan-devkit API获取数据集中的scenarios ①

```
scenario.get_ego_past_trajectory()
scenario.get_past_tracked_objects()
scenario.get_ego_future_trajectory()
scenario.get_future_tracked_objects()
scenario.get_route_roadblock_ids()
```

处理scenarios中的agent和地图数据 ②

```
class GraphDataset(torch_geometric.data.InMemoryDataset):
    def __init__(self, root, transform=None, pre_transform=None):
        super(GraphDataset, self).__init__(root, transform, pre_transform)
        self.data, self.slices = torch.load(self.processed_paths[0])

    def process(self):
        data_path_list = [...]
        graph_data_list = []
        for data_path in data_path_list:
            graph_data = torch_geometric.data.Data(...)
            graph_data_list.append(graph_data)
        data, slices = self.collate(graph_data_list)
        torch.save((data, slices), self.processed_paths[0])
```

创建Pytorch Geometric可直接加载的数据集 ③

TIPS: 除了nuPlan官方文档外，还可参考DTTP^[1]、GameFormer^[2]等代码中有关nuPlan的处理

[1] <https://github.com/MCZhi/DTTP>

[2] <https://github.com/MCZhi/GameFormer>

在开始训练前，需要先从nuplan数据中提取我们需要的数据批量处理后得到vectornet训练所需的数据集，使用nuplan-devkit提供的API提取场景，这里的get_filter_parameters实际上就是在设置筛选需要哪些场景的数据，比如直行左转、直行右转、大曲率弯道、大幅度加减速等，get_scenarios会返回全部符合条件的场景变量。根据场景类提供的接口就可以拿到场景我们关心的数据，像地图元素、agent轨迹等，其中关键的其实就这五个，从函数名也很好猜测他们的作用，分别是获取自车和他车的历史轨迹，未来轨迹，和地图元素，不过这还只是原始数据需要我们根据前面讲的向量化特征表示方法转换为vectornet网络的输入形式。最后还需要将这些数据转换为torch_geometric pytorch图神经网络模块能直接加载的形式，pytorch提供了InMemoryDataset类，我们继承并重载process函数即可。在process函数中，我们需要将数据转换成torch_geometric.data.Data的变量类型保存，声明输入的节点、边、和标签真值等，torch.save最后会将数据保存为一个.pt文件，在训练时就可以直接加载了。

以上是关于训练数据处理的部分，在实际运行做前向推理时的过程基本一致，不过拿到的不是scenario这个包含当前全部信息的对象了，获取信息的api会发生些变换，但也都大同小异。

这里有个小提示就是，关于nuplan数据的交互除了看官方文档外，还可以参考一些开源基于nuplan做预测决策的代码，如DTTP、GameFormer等，这样能够帮助我们更快上手nuplan。



感谢各位聆听 !
Thanks for Listening



好，以上就是本次分享的全部内容了，谢谢大家